Reproducing Dobi-SVD: A Reproduction and Evaluating Its Performance on Different Models

Кабиров Руслан
kabirov.ra@phystech.edu

Кабиров Рустам
kabirov.rustam@phystech.edu

Илья Овсеенко
ovseenko.is@phystech.edu

Заходов Егор
zakhodov.ea@phystech.edu

April 26, 2026

В статье [1] предложен метод сжатия больших языковых моделей на основе сингулярного разложения (SVD), называемый Dobi-SVD. В данной работе мы воспроизводим эксперименты оригинальной статьи и проводим дополнительную эмпирическую оценку метода на различных языковых моделях. Также сравниваем этот метод с менее актуальными методами, использующие SVD разложение. Анализируем качество моделей в зависимости от коэффициента сжатия.

1 Идея

В рамках данного проекта мы воспроизведем метод сжатия больших языковых моделей на основе SVD-разложения, предложенный в работе [1]. Этот метод демонстрирует превосходство над activation-aware подходами, представленными в статьях [2] и [3].

Проект включает следующие этапы: повторение экспериментов из оригинальной статьи и проведение дополнительной оценки метода и по возможности на других языковых моделях квантизованных вариантов LLaMa-3 с учетом квантизации. Мы надеемся, что глубокое изучение метода в процессе его реализации позволит выявить новые идеи для дальнейшего развития подхода.

1.1 Problem

Инференс современных языковых моделей требует значительных вычислительных ресурсов, поэтому в последние годы активно исследуются методы их сжатия. К наиболее распространенным подходам относятся квантизация и прунинг.

Квантизация уменьшает объем используемой памяти и ускоряет вычисления за счет замены операций с числами с плавающей точкой на более дешевые integer-операции. Однако такой подход не всегда является достаточно гибким и часто оказывается сильно привязан к конкретному аппаратному обеспечению.

Прунинг, в свою очередь, сокращает размер модели за счет удаления параметров, которые считаются наименее важными для ее работы. Несмотря на простоту этой идеи, на практике прунинг нередко приводит к заметной деградации качества модели.

Отдельное направление методов сжатия основано на низкоранговых аппроксимациях матриц весов с помощью SVD. В базовом варианте такая постановка сводится к задаче

$$W_k^\star = \arg\min_{W_k} \|W - W_k\|_F^2.$$

Иными словами, метод пытается найти низкоранговую матрицу W_k, которая как можно точнее приближает исходную матрицу весов W. Однако для языковых моделей такая постановка оказывается недостаточной: хорошее приближение самой матрицы весов не гарантирует сохранения качества модели, поскольку важно не только значение весов, но и то, как они действуют на реальные входные активации.

Следующим шагом стали activation-aware подходы, такие как ASVD и SVD-LLM. В них оптимизируется уже не расстояние между матрицами весов, а расстояние между выходами слоя на реальных активациях:

$$W_k^\star = \arg\min_{W_k} \|WX - W_k X\|_F^2.$$

Такая постановка лучше отражает поведение слоя внутри модели, поскольку учитывает calibration activations X. Более того, для этой задачи можно получить аналитическое решение. Тем не менее локальная оптимальность на уровне одного слоя не гарантирует оптимального значения итогового task loss всей модели. Поэтому методы, основанные только на такой локальной аппроксимации, часто уступают более сильным современным подходам, включая SOTA-квантизацию, прунинг и Dobi-SVD.

В работе [1] авторы предлагают другой взгляд на SVD-сжатие. Вместо того чтобы заранее задавать ранг низкоранговой аппроксимации, они делают ранг обучаемым параметром. Для этого используется calibration dataset: модель прогоняется на этом наборе данных, после чего для сжимаемых слоев собираются активации A.

Далее для каждой активации выполняется SVD-разложение:

$$A = U_A \Sigma_A V_A^T,$$

где

$$\Sigma_A = \mathrm{diag}(\sigma_1, \sigma_2, \ldots, \sigma_n).$$

В обычном truncated SVD нужно жестко выбрать, сколько сингулярных значений оставить. Такая операция дискретна и не позволяет напрямую обучать ранг градиентными методами. Dobi-SVD заменяет жесткое отсечение на гладкое:

$$\Sigma_k = \mathrm{diag}\big(T(\sigma_1), \ldots, T(\sigma_n)\big),$$

где

$$T(\sigma_i) = \sigma_i \left(0.5 \tanh(\beta(k-i)) + 0.5\right).$$

Здесь k является обучаемым параметром, задающим мягкую границу отсечения сингулярных значений, а β управляет резкостью этого перехода. Если i заметно меньше k, соответствующее сингулярное значение почти сохраняется; если i заметно больше k, оно почти зануляется.

После этого восстанавливается приближенная активация:

$$\widetilde{A} = U_A \Sigma_k V_A^T.$$

На этапе оптимизации все веса модели замораживаются, и обучаемыми остаются только значения k для разных матриц. Для их подбора используется multi-objective loss:

$$L = L_{task} + \gamma \left|R_{now} - R_{tar}\right|.$$

Первое слагаемое, L_{task}, отвечает за сохранение качества модели на целевой задаче. Второе слагаемое штрафует отклонение текущей степени сжатия R_{now} от заданной целевой степени сжатия R_{tar}. Таким образом, Dobi-SVD подбирает ранги не вручную и не одинаково для всех слоев, а адаптивно: важные матрицы могут сохранить больший ранг, тогда как менее чувствительные матрицы сжимаются сильнее.

Именно это делает Dobi-SVD более гибким по сравнению с классическими SVD-подходами. Метод оптимизирует не только локальную ошибку аппроксимации весов или выходов отдельного слоя, а подбирает степень сжатия с учетом итогового качества всей модели.

2 Literature review

2.1 ASVD

На практике матрицы весов в языковых моделях сжимаются не одинаково хорошо. У некоторых матриц сингулярные значения распределены так, что несколько первых значений сильно доминируют над остальными. Такие матрицы обычно лучше переносят низкоранговое приближение. У других матриц сингулярные значения распределены более равномерно, поэтому их агрессивное сжатие быстрее приводит к потере качества модели. Из-за этого один и тот же наивный SVD-подход плохо подходит для всех слоев сразу: желательно учитывать не только сами веса, но и то, какие входные активации проходят через соответствующую матрицу.

В работе [2] авторы предлагают метод ASVD, который решает задачу

$$W_k^\star = \arg\min_{W_k} \|WX - W_kX\|_F^2.$$

Здесь цель состоит не просто в том, чтобы приблизить матрицу весов W, а в том, чтобы сохранить результат ее действия на реальные входные активации X.

Идея метода состоит в следующем:

1. Сначала матрица весов масштабируется:

$$W' = WS,$$

   где S — обратимая матрица.

2. Затем к преобразованной матрице W' применяется усеченное SVD-разложение:

$$W' \approx U_k'\Sigma_k'V_k'^T.$$

3. После этого масштабирование компенсируется обратно:

$$V_k''^T = V_k'^T S^{-1}.$$

   В итоге приближение исходной матрицы имеет вид

$$W_k = (U_k'\Sigma_k'V_k'^T)S^{-1} = U_k'\Sigma_k'V_k''^T.$$

4. Матрица S выбирается на основе calibration set. Для этого набор данных прогоняется через модель, после чего собираются активации X. В ASVD используется диагональная матрица S, элементы которой задаются как

$$S_{ii} = \left(\frac{1}{n}\sum_{j=1}^{n}|X_{ij}|\right)^\alpha.$$

   Параметр α является гиперпараметром; в оригинальной статье авторы используют значение $\alpha = 0.5$.

Таким образом, ASVD учитывает важность отдельных входных каналов: если соответствующая активация в среднем имеет большую величину, то этот канал масштабируется сильнее и лучше сохраняется при низкоранговом приближении. Это делает сжатие менее “слепым” по сравнению с обычным SVD и позволяет уменьшить потери качества.

Однако у метода есть существенный недостаток. При достаточно сильном сжатии качество модели все равно быстро деградирует. Поэтому ASVD хорошо работает только при умеренных степенях компрессии, тогда как при более агрессивном сжатии Dobi-SVD показывает себя лучше.

2.2 SVD-LLM

В отличие от наивного SVD-сжатия, которое просто отбрасывает малые сингулярные значения, SVD-LLM [3] сначала подготавливает данные так, чтобы усечение ранга лучше соответствовало реальной ошибке аппроксимации. Для этого на калибровочном наборе строится whitening matrix S, где $X \in \mathbb{R}^{n\times d}$ обозначает матрицу активаций, собранных на входах слоя, а $W \in \mathbb{R}^{m\times d}$ — матрицу весов линейного слоя.

Сначала входные активации нормализуются:

$$\widetilde{X} = S^{-1}X,$$

так что каналы становятся менее скоррелированными. Идея состоит в том, чтобы добиться приближённой ортогональности

$$\widetilde{X}\widetilde{X}^\top \approx I.$$

Затем веса преобразуются в whitened space:

$$W' = WS.$$

После этого к матрице W' применяется SVD:

$$W' \approx U_k \Sigma_k V_k^\top,$$

Итоговая аппроксимация исходной матрицы восстанавливается как

$$W_k = U_k \Sigma_k V_k^\top S^{-1}.$$

Таким образом, SVD-LLM не просто делает low-rank approximation, а сначала изменяет пространство признаков, чтобы truncation по сингулярным значениям был более согласован с потерей качества.

Дополнительно метод использует layer-wise closed-form update: после усечения ранга оставшиеся параметры слоя пересчитываются аналитически, чтобы компенсировать деградацию, возникшую после SVD-truncation. Это позволяет избежать полного дообучения модели и лучше сохранять качество при высоких степенях сжатия.

2.3 SVD-LLM V.2

Метод SVD-LLM показал высокую эффективность для post-training сжатия больших языковых моделей с помощью сингулярного разложения весовых матриц. Однако он имеет ряд ограничений, снижающих качество сжатия на практике.

Во-первых, SVD-LLM использует разложение Холецкого для учёта входных активаций. Этот шаг требует положительной определённости соответствующей матрицы, что не всегда выполняется для высокоразмерных и плохо обусловленных матриц активаций. Кроме того, численная нестабильность разложения может приводить к росту фактических потерь усечения.

Во-вторых, SVD-LLM применяет одинаковый коэффициент сжатия ко всем весовым матрицам. Такой подход не учитывает, что разные слои и типы матриц имеют различную чувствительность к SVD-усечению и могут по-разному влиять на truncation loss. Поэтому однородное сжатие может приводить к неоптимальному распределению ошибки и ухудшению качества модели.

Метод SVD-LLM V2 [4] решает эти проблемы за счёт двух ключевых изменений. Во-первых, он оценивает теоретический truncation loss для каждой весовой матрицы и на его основе назначает индивидуальный коэффициент сжатия: более чувствительные матрицы сжимаются слабее, а более избыточные - сильнее. Во-вторых, SVD-LLM V2 заменяет разложение Холецкого на более устойчивую процедуру, основанную на SVD, что позволяет избежать требования положительной определённости и повысить численную стабильность сжатия.

Прямого сравнения Dobi-SVD и SVD-LLM V2 в доступной литературе обнаружить не удалось. Поэтому дальнейшее экспериментальное сравнение этих подходов представляет интерес для выявления их преимуществ, ограничений и применимости к различным архитектурам LLM

2.4 AdaSVD

В отличие от методов с фиксированным rank truncation, AdaSVD [5] подбирает степень сжатия адаптивно для разных матриц и слоёв. Идея состоит в том, что разные веса в LLM имеют различную спектральную структуру: в одних матрицах большая часть энергии сосредоточена в нескольких первых сингулярных значениях, а в других она распределена более равномерно. Поэтому один и тот же уровень усечения не является оптимальным для всех слоёв.

Пусть $W \in \mathbb{R}^{m \times d}$ — матрица весов линейного слоя. Тогда AdaSVD строит low-rank approximation вида

$$W \approx U_k \Sigma_k V_k^\top,$$

где ранг k выбирается не заранее, а в зависимости от спектральной структуры конкретной матрицы и заданного бюджета сжатия. Таким образом, метод старается сохранять больше информации в тех слоях, где она действительно сосредоточена, и сильнее сжимать менее чувствительные слои.

Если обозначить ошибку аппроксимации как

$$\mathcal{E}(k) = \|W - U_k \Sigma_k V_k^\top\|_F^2,$$

то AdaSVD выбирает такой k, который обеспечивает компромисс между малой ошибкой и требуемой степенью компрессии. На практике это делает метод более гибким, чем глобальное фиксированное усечение ранга, и позволяет лучше учитывать различия между слоями.

По результатам статьи, AdaSVD показывает, что адаптивный выбор ранга позволяет стабильнее сохранять качество модели при сильном сжатии по сравнению с одинаковым усечением для всех матриц.

2.5 RES-SVD

RES-SVD[6] - это метод SVD-сжатия LLM, который делает две главные вещи:

1. Компенсация ошибки усечения через остаточную матрицу. Сначала матрицу весов сжимают обычным SVD до промежуточного ранга. Затем вычисляют остаток

$$R = W - W_{r_1},$$

   после чего этот остаток также аппроксимируют с помощью SVD и добавляют обратно.

2. Сжатие только последних слоёв модели. В статье описывающей этот метод авторы показывают, что при сильном сжатии ранних слоёв ошибка начинает накапливаться. Поэтому в ResSVD выборочно сжимают только последние несколько слоёв, чтобы уменьшить распространение ошибки.

По экспериментальным результатам Dobi-SVD превосходит ResSVD на сопоставимых уровнях сжатия. Например, при среднем сжатии средняя accuracy у Dobi-SVD составляет 0.47 против 0.39 у ResSVD, а при сильном сжатии — 0.40 против 0.32. Кроме того, Dobi-SVD даёт значительно меньшую perplexity, например при сильном сжатии на WikiText-2: 9.95 против 48.67, что указывает на лучшую устойчивость метода к агрессивному сжатию.

3 Метрики качества

В рассматриваемой нами статье используются следующие метрики качества:

- Perplexity (PPL) на датасетах Wikitext-2, C4, PTB
- Accuracy на in-domain и cross-domain задачах
- Throughput инференса (tokens/sec)
- Latency инференса (задержка на запрос)
- Устойчивость качества при высоких коэффициентах сжатия (через PPL и accuracy как косвенная метрика)
- Truncation loss - фробениусова норма разности активации и ее приближения

4 План

- Воспроизвести эксперименты Dobi-SVD на моделях LLaMA-2-7B, LLaMA-3-8B и LLaMA-3.1-8B без применения квантизации. Для этого планируется варьировать коэффициент сжатия и запускать основные базовые методы, рассмотренные в статье, с целью проверки воспроизводимости результатов и проведения сопоставимого анализа.
- Провести эксперименты по сжатию моделей с использованием SVD-LLM V2 и RES-SVD на тех же архитектурах и датасетах, что и в экспериментах с Dobi-SVD. Это позволит выполнить прямое сравнение данных подходов, поскольку в существующей литературе они не были сопоставлены между собой в единой экспериментальной постановке.
- При наличии вычислительных ресурсов и времени дополнительно исследовать совместимость SVD-based методов с квантизацией. В частности, планируется оценить, как применение квантизации после SVD-сжатия влияет на качество модели, степень сжатия и эффективность инференса.
- Проанализировать возможные модификации рассматриваемых методов и предложить собственные варианты улучшения. Полученные модификации планируется сравнить с воспроизведёнными базовыми результатами для оценки их эффективности.

5 Outcomes

По итогам работы будем иметь такие результаты:

- Таблицы с метриками, полученными при сжатии моделей LLaMa-2-7B, LLaMa-3-8B, LLaMa-3.1-8B с помощью Dobi-SVD
- Сравнения качества сжатия между методами Res-SVD, SVD-LLM V.2 и Dobi-SVD
- Возможно модели, на которых Dobi-SVD уступает другим методам
- Набор чекпойнтов - набор весов модели с разным коэффициентом сжатия (0.4, 0.6, 0.8), которые можно использовать для экспериментов с инференсом.
- Возможно применение Dobi-SVD к квантизованным версиям ранее упомянутых моделей.

References

[1] Qinsi Wang, Jinghan Ke, Masayoshi Tomizuka, Yiran Chen, Kurt Keutzer, and Chenfeng Xu. Dobi-svd: Differentiable svd for llm compression and some new perspectives, 2025.

[2] Zhihang Yuan, Yuzhang Shang, Yue Song, Dawei Yang, Qiang Wu, Yan Yan, and Guangyu Sun. Asvd: Activation-aware singular value decomposition for compressing large language models, 2025.

[3] Xin Wang, Yu Zheng, Zhongwei Wan, and Mi Zhang. Svd-llm: Truncation-aware singular value decomposition for large language model compression, 2025.

[4] Xin Wang, Samiul Alam, Zhongwei Wan, Hui Shen, and Mi Zhang. Svd-llm v2: Optimizing singular value truncation for large language model compression, 2025.

[5] Zhiteng Li, Mingyuan Xia, Jingyuan Zhang, Zheng Hui, Haotong Qin, Linghe Kong, Yulun Zhang, and Xiaokang Yang. Adasvd: Adaptive singular value decomposition for large language models. *arXiv preprint arXiv:2502.01403*, 2025.

[6] Haolei Bai, Siyong Jian, Tuo Liang, Yu Yin, and Huan Wang. Erc-svd: Error-controlled svd for large language model compression, 2026.